

OPERATION PHANTOM SHELL

Linux Hackathon — Official Problem Statement

Duration: 3 Hours |

BEFORE YOU BEGIN

Step 1: Open your browser → shell.cloud.google.com

Step 2: Sign in with your Google account. Cloud Shell launches automatically. No installation. No setup.

Step 3: Read each challenge completely before you start writing a single command.

Step 4: AI tools are permitted. Whether they help or hurt you depends entirely on how well you understand what they give you.

RULES

- 1. **Bash only.** No Python, no Node.js, no Ruby — unless a challenge explicitly states otherwise. Standard Linux utilities only.
 - 2. **Google Cloud Shell only.** No local machine, no external servers.
 - 3. **Every answer must be verifiable.** Showing code is not enough. Show the output. Show it runs correctly.
 - 4. **No hardcoded values** that are supposed to be computed. Judges will modify inputs and re-run your solution.
 - 5. **Partial submissions are accepted** per challenge. You get nothing for a challenge you don't attempt.
-

SCORING

#	Challenge	Points	Bonus
1	The Log Analyst	15	+5
2	The Process Wrangler	20	+5
3	The Network Detective	15	+5
4	The Filesystem Archaeologist	20	+5
5	The Final Boss	30	+10

Base Total: 100 points | Maximum with Bonus: 130 points

Bonus conditions are listed at the end of each challenge. They are strict. Either you meet them exactly or you don't.

CHALLENGE 1 — THE LOG ANALYST

"20 files. 20,000 lines. One truth."

Points: 15 | Bonus: +5

Part A — Generate the Data

Write a **single bash script** that creates the following structure:

```
~/phantom/
└─ logs/
    ├── app_1.log
    ├── app_2.log
    ├── ...
    └─ app_20.log
```

Each file must contain exactly **1000 lines**. Every single line across all files must strictly follow this format:

```
[YYYY-MM-DD HH:MM:SS] [LEVEL] [service-name] message text here
```

Constraints:

- Timestamps must be random but chronologically valid dates within the year 2024
 - `LEVEL` must be one of exactly four values: `INFO`, `WARN`, `ERROR`, `DEBUG` — distributed randomly
 - Service names must be chosen randomly from a pool of at least 8 names that you define yourself
 - Message text must not repeat — every line must be distinct
 - **Running your script twice must produce the exact same output both times.** This is a hard requirement, not a suggestion.
 - No Python. No external tools. Bash and standard Linux utilities only.
-

Part B — Query the Data

Using only **command-line one-liners** (no scripts for this part), produce the following answers from the data your script generated:

Query 1: List the top 3 services ranked by total number of `ERROR` entries across all 20 files. Show service name and count.

Query 2: Identify every clock-minute (`YYYY-MM-DD HH:MM`) during which two or more **different** services each logged at least one `ERROR`. For each such minute, show the minute and the count of distinct services that errored.

Query 3: Which single hour of the day (expressed as 0–23) had the greatest total number of log lines across all 20 files combined?

You must show every command used. The output must be consistent with your generated data.

Bonus (+5 points)

Your Part A script is **20 lines or fewer**. Blank lines and comment lines do not count toward the limit.

CHALLENGE 2 — THE PROCESS WRANGLER

"Spawn it. Watch it. Recover it. Kill it clean."

Points: 20 | Bonus: +5

The Task

Write a single bash script named `phantom_monitor.sh`.

Worker Requirements

When the script runs, it must spawn exactly **5 background worker processes**. Each worker must:

- Run indefinitely in a loop
 - Sleep a random duration between 2 and 8 seconds per iteration
 - After each sleep, append exactly one line to `/tmp/worker_[ID].log` in this format:
[YYYY-MM-DD HH:MM:SS] worker-[ID] heartbeat [count]
 - `count` begins at 1 and increments by 1 with every heartbeat for that worker
 - Worker IDs are 1 through 5
-

Monitor Requirements

The monitor process (the main script) must:

- Know the PID of every worker at all times
- Print a status report to the terminal **every 10 seconds** in this format:

```
---- PHANTOM MONITOR STATUS [YYYY-MM-DD HH:MM:SS] ----
PID      WORKER-ID    HEARTBEATS    LAST-SEEN
[pid]    worker-1      [n]           [timestamp]
[pid]    worker-2      [n]           [timestamp]
[pid]    worker-3      [n]           [timestamp]
[pid]    worker-4      [n]           [timestamp]
[pid]    worker-5      [n]           [timestamp]
```

Shutdown Requirements

When the script receives `SIGTERM` or `SIGINT` (Ctrl+C):

- All 5 workers must be stopped immediately
 - The monitor must print a shutdown confirmation message
 - The script must exit
 - **After exit, no process spawned by this script may remain running**
-

The Hard Constraint

Every piece of logic — workers, monitor, status reporting, restart detection, signal handling — must exist inside **one single bash script**. No helper files. No sourced scripts.

Verification — You Must Demonstrate All of the Following

1. Run the script. Let it run for at least **60 seconds**. Show a minimum of 2 status reports printed to terminal.

2. While running, manually kill one worker using `kill [PID]`. Show the monitor detecting the death and restarting it within 5 seconds.
3. Press Ctrl+C to trigger shutdown.
4. Immediately after exit, run both of these commands and show their output:
5. `ps aux | grep phantom_monitorps aux | grep worker`

Neither must show any live processes from your script.

Bonus (+5 points)

After shutdown, run `ps tree -p` and show that **zero** child processes originating from your script appear anywhere in the output.

CHALLENGE 3 — THE NETWORK DETECTIVE

"Build the request. Parse the response. Do it bare-handed."

Points: 15 | Bonus: +5

Part A — The Crafted Request

Send a single HTTP request to:

`https://httpbin.org/anything`

The request must meet **all** of the following specifications simultaneously:

Method: POST

Custom Header — computed, not hardcoded:

X-Phantom-Token: <base64 encoding of the string "phantom_YYYYMMDD" using today's date>

Request Body — all three fields computed, not hardcoded:

```
{
  "username": "<your Cloud Shell username>",
  "timestamp": "<current UTC time as Unix epoch integer>",
  "checksum": "<SHA256 hash of the exact string: phantom_secret_2024>"
}
```

Show the complete `curl` command you used. Every dynamic value must be computed inline within the command itself.

Part B — Parse the Response

`httpbin.org/anything` echoes everything you sent back in its JSON response.

Using **only** `grep`, `awk`, `sed`, and `tr` — no Python, no `jq`, no other tools — extract and print the following from the raw response:

1. The value of your `X-Phantom-Token` header as the server received it
2. The `Content-Type` of the response
3. The `origin` IP address from the response body

Show every command used to extract each value.

Part C — The One-Liner

Write a **single one-line bash command** that does all of the following:

- Sends a GET request to `https://httpbin.org/uuid` exactly **10 times**, in **parallel**
- Extracts the UUID value from each response
- Sorts all 10 UUIDs alphabetically
- Prints only the UUIDs where the **first 8 characters** contain at least one occurrence of the letter `a`

The entire thing — requests, extraction, filtering, sorting — must be one line. No semicolons to chain separate commands into a fake one-liner. It must genuinely be a single pipeline or compound expression.

Bonus (+5 points)

Your Part C one-liner is **100 characters or fewer** in total length.

CHALLENGE 4 — THE FILESYSTEM ARCHAEOLOGIST

"Create it. Destroy selectively. Read it without opening it."

Points: 20 | Bonus: +5

Part A — Create the Archive

Create the directory `~/phantom/archive`. Inside it, generate exactly **100 files** named `record_001.dat` through `record_100.dat`.

Requirements:

- Each file must contain exactly **512 bytes** of pseudo-random data
 - Accomplish this in a **maximum of 3 lines** of bash
-

Part B — Selective Deletion

Delete every file whose number is a **prime number**. So `record_002.dat`, `record_003.dat`, `record_005.dat`, `record_007.dat` and so on must be deleted.

Requirements:

- This must be done as a **single bash one-liner**
 - Your one-liner must **compute the primes programmatically** — no hardcoded list of primes, no pre-written arrays
 - After execution, verify the correct files were deleted by showing the count of remaining files (there are exactly 25 primes between 1 and 100)
-

Part C — Inspect Without Extracting

Compress all remaining files into a single archive:

```
~/phantom/phantom_archive.tar.gz
```

Then, **without extracting the archive**, answer all four of the following using only command-line tools operating directly on the `.tar.gz` file:

1. How many files are inside the archive?
2. What is the total uncompressed size of all files inside?

3. Which file inside the archive is the largest?
4. What is the MD5 hash of `record_004.dat` as it exists inside the archive?

Show every command used. Answers must be consistent with your archive.

Part D — The Manifest Script

Write a bash script called `manifest.sh` that:

- Takes any `.tar.gz` file as its only argument
- Outputs one line per file inside the archive in this format:
[filename] [uncompressed-size-in-bytes] [md5hash]
- Works by operating on the archive directly — it must **not** fully extract the archive to disk at any point

Test it against your `phantom_archive.tar.gz` and show the output.

Bonus (+5 points)

Your `manifest.sh` detects and handles a corrupt or invalid `.tar.gz` file gracefully — printing a clear error message and exiting with a non-zero exit code instead of crashing or producing garbage output. Demonstrate this by running it against a deliberately corrupted file.

CHALLENGE 5 — THE FINAL BOSS

"Build a load tester. From scratch. In bash."

Points: 30 | Bonus: +10

The Task

Build a fully functional HTTP load testing tool as a single bash script called `phantom_load.sh`. No Python. No Node.js. No external tools beyond standard Linux utilities and `curl`.

Interface

Your script must accept the following command-line flags:

Flag	Description
-u URL	Target URL to test
-n NUMBER	Total number of requests to send
-c CONCURRENCY	Maximum number of parallel requests at any given time
-m METHOD	HTTP method: GET or POST
-t TIMEOUT	Per-request timeout in seconds

Example invocation:

```
./phantom_load.sh -u https://httpbin.org/delay/1 -n 50 -c 10 -m GET -t 5
```

Behaviour Requirements

- At no point may more than `-c` requests be running simultaneously. This is a hard concurrency cap, not a suggestion.
 - Requests must begin as soon as a concurrency slot opens — no waiting for entire batches to finish before starting the next batch.
 - Every request's response time must be measured in milliseconds.
 - HTTP responses must be classified as success (2xx status code) or failure (non-2xx or timeout).
-

Output Report

Upon completion, the script must print a report in exactly this format:

```
===== PHANTOM LOAD TEST REPORT =====
Target URL      : https://httpbin.org/delay/1
HTTP Method     : GET
Total Requests  : 50
Concurrency     : 10
Timeout (s)     : 5
-----
Results
-----
Successful      : [count]
Failed          : [count]
-----
Response Times (ms)
-----
Minimum         : [value]
Maximum         : [value]
Average         : [value]
P95             : [value]
-----
Throughput      : [value] requests/second
=====
```

Constraints

- `Successful + Failed` must equal `Total Requests` exactly
 - P95 must be calculated correctly — it is the value below which 95% of response times fall, computed from the actual collected timings
 - Throughput must be calculated from total wall-clock time, not estimated
 - The script must clean up all temporary files it creates, regardless of how it exits
-

Verification

Run your script with exactly these parameters:

```
./phantom_load.sh -u https://httpbin.org/delay/1 -n 50 -c 10 -m GET -t 5
```

Show the complete output report. The numbers must be internally consistent. Judges will re-run your script independently.

Bonus (+10 points)

Your P95 value is within **2% of the ground truth** P95 calculated by independently timing all 50 requests. Judges will verify this by running your script and computing P95 independently. No approximations. The math must be correct.

SUBMISSION FORMAT

For each challenge, submit the following in order:

- 1. Your Code** Paste the complete bash script(s) exactly as they are on your Cloud Shell.
- 2. Verification Output** Paste the exact terminal output that proves your solution works. Screenshots are not accepted. Text output only.
- 3. Bonus Claim (if applicable)** If you are claiming bonus points, explicitly state it and show the specific output that proves the bonus condition is met.

Incomplete submissions are graded on what is shown. If you can't demonstrate it works, it doesn't work.

FINAL NOTE

Every challenge in this hackathon has a correct solution and an almost-correct solution. The almost-correct solution looks right, runs without errors, and produces plausible output. It is wrong. The gap between almost-correct and correct is exactly what this hackathon is testing.

Good luck. You'll need it.

Operation Phantom Shell | Linux Hackathon